

SYSTEM GUIDE · PLAIN-LANGUAGE EDITION

OTIF Risk Intelligence

A control-tower prototype that predicts, explains, and helps prioritize the response to orders at risk of missing an **On-Time-In-Full (OTIF)** delivery promise.

What this document is

This guide explains the OTIF Risk Intelligence system from the ground up, for anyone — including readers with no background in data science, statistics, or software engineering. Every technical term is defined the first time it is used. The goal is that after reading this document, you should be able to explain to a colleague what the system does, how its data is produced, how its predictions are made and explained, how it decides what action to recommend, how its performance is measured, and — just as importantly — what it does **not** do.

One-paragraph summary

The system watches open customer orders moving through a supply chain (a vendor ships goods, a distribution center picks and packs them, a truck carries them, a customer receives them) and asks one question about each order: is it at risk of arriving late, short on quantity, or both? It answers using a statistical model trained on a carefully built practice dataset (a **digital twin**, explained in Section 4), explains its reasoning in plain terms, estimates which specific products on the order are likely affected, checks whether the people and equipment needed to fix the problem are actually available, and recommends one clear action. A second, simpler causal-style model cross-checks the first and can also test "what if we fixed this cause?" style scenarios, honestly labeled as scenario analysis rather than a proven cure. Everything the system decides is logged, monitored over time for signs of decay, and only ever put into service after passing a documented set of checks — a person, or another governance process, is always in the loop before anything is acted upon in the real world. The entire system today runs on realistic *simulated* data, not live company systems, precisely so its behavior can be measured, challenged, and understood before anyone considers connecting it to a real warehouse.

Document map

This guide is organized in the same order a new reader would naturally need to understand the system: what problem it solves, how its practice data is generated, how it turns that data into a prediction and an explanation, how it decides what to do about it, how it is kept honest over time, and finally what is genuinely real versus simulated versus not yet built. A full table of contents follows on the next page.

Table of Contents

1	Executive summary
2	What "OTIF" means, with examples
3	End-to-end architecture
4	The practice data: a noisy digital twin
5	The ten data tables, explained
6	Data validation: catching bad data before it spreads
7	Point-in-time scoring and data leakage
8	Turning raw data into features
9	Why the data is split by time, not at random
10	The risk model (XGBoost), in plain language
11	Explaining a single score (SHAP)
12	Which products on the order are affected
13	The 10-node cause-and-effect network
14	"What if we fixed this?" scenario analysis
15	Combining the two models (fusion)
16	Turning a score into a decision
17	The Decision Value Lab
18	Exploration: keeping some capacity for learning
19	A day in the life: the operations replay
20	Production-shaped adapters and service contracts
21	Manifests, lineage, and tamper detection
22	The decision ledger and observed outcomes
23	Monitoring, drift, and service targets
24	Champion/challenger governance
25	The AI Copilot
26	The nine screens of the control tower
27	One order, start to finish
28	What is real, simulated, and not yet built
29	Results and limitations, in one place
30	How to run it
31	Glossary
32	Appendix: module map and artifact directory map

1. Executive summary

1.1 The problem

A company that ships physical goods makes a promise to every customer: the order will arrive *on time* and *complete*. Missing that promise — arriving late, arriving short on quantity, or both — is called an **OTIF miss** (On-Time-In-Full miss). OTIF misses cost money (contractual penalties, expedited shipping, customer goodwill) and, because a supply chain has many moving parts (vendors, warehouses, trucks, customers), it is often unclear in advance which orders are at risk and why. By the time a miss is obvious, it is usually too late to do anything about it.

1.2 What the system does

OTIF Risk Intelligence is a prototype "control tower" that, for every open order, produces one risk score, one plain-language explanation, an estimate of which specific products on the order are affected, a check on whether the resources needed to fix the problem are actually available, and one recommended action. It also runs a simulated day-by-day operating loop so its behavior over weeks and months can be observed, tracks whether its own predictions are decaying, and only ever promotes an improved version of itself after it passes a documented set of checks.

1.3 What the system does not do

- It does **not** connect to a real company's live ordering, warehouse, or shipping systems today. All data is generated by a realistic simulator (Section 4).
- It does **not** prove that any recommended action causes an improvement in the real world. Everything labeled a "cause" or an "intervention effect" in this document is either an association learned from data, or a scenario computed inside a simulated practice environment — never an observed real-world experiment.
- It does **not** make decisions autonomously. Every recommendation is meant for a human planner to review, accept, reject, or override; the optional AI assistant (Section 25) can only explain and draft text — it cannot change a score, a decision, or an allocation.
- It does **not** claim production readiness. Held-out accuracy numbers measured on simulated data are not evidence that the system would perform the same way on a real supply chain.

In one sentence: this is a working, well-tested, and honestly self-reported prototype that demonstrates every piece a real OTIF risk desk would need — prediction, explanation, resource-aware decisioning, monitoring, and governance — built and measured end to end on realistic synthetic data.

2. What "OTIF" means, with examples

OTIF stands for **On-Time, In-Full**. An order "hits OTIF" only if *both* of the following are true:

- **On time**: the order is delivered at or before the promised delivery date.
- **In full**: the customer receives the full quantity that was ordered — not a partial shipment.

Because there are two independent conditions, there are exactly four possible outcomes for any order:

On time?	In full?	Outcome	Everyday example
Yes	Yes	OTIF success	1,000 units arrive on the promised date, all 1,000 units present. No penalty, customer expectation met.
Yes	No	OTIF miss — quantity failure	The truck arrives exactly on the promised date, but only 850 of the 1,000 units are on it because a warehouse ran short. Arrived on schedule, but incomplete.
No	Yes	OTIF miss — timing failure	All 1,000 units eventually arrive, but two days after the promised date because of a carrier delay. Complete, but late.
No	No	OTIF miss — both	The truck arrives three days late <i>and</i> only 700 of the 1,000 units are on it. Both promises broken at once.

This system treats "OTIF miss" as the single outcome it ultimately tries to predict, but — importantly — it does not treat lateness and incompleteness as one blended idea. Internally (Section 13) it keeps a separate notion of an **in-full failure** (quantity problem) and a **late-delivery failure** (timing problem), because in this system's practice data, most OTIF misses turn out to be quantity-driven, not timing-driven — an important, measured finding revisited in Section 13.

Why this distinction matters operationally: the fix for "we will not have enough units" (expedite inventory, reallocate from another location) is different from the fix for "the truck will be late" (use an alternate carrier, coordinate a new delivery appointment). A system that only predicts one blended "will this order fail" number cannot tell a planner which of these two very different problems to solve.

3. End-to-end architecture

The diagram below shows every major stage the system passes through, from generating practice data to recommending — and eventually governing — a real action. Solid arrows are the path a live prediction takes. The dashed arrow shows that "what-if" scenario analysis is informational only: it never changes the score or the decision above it.

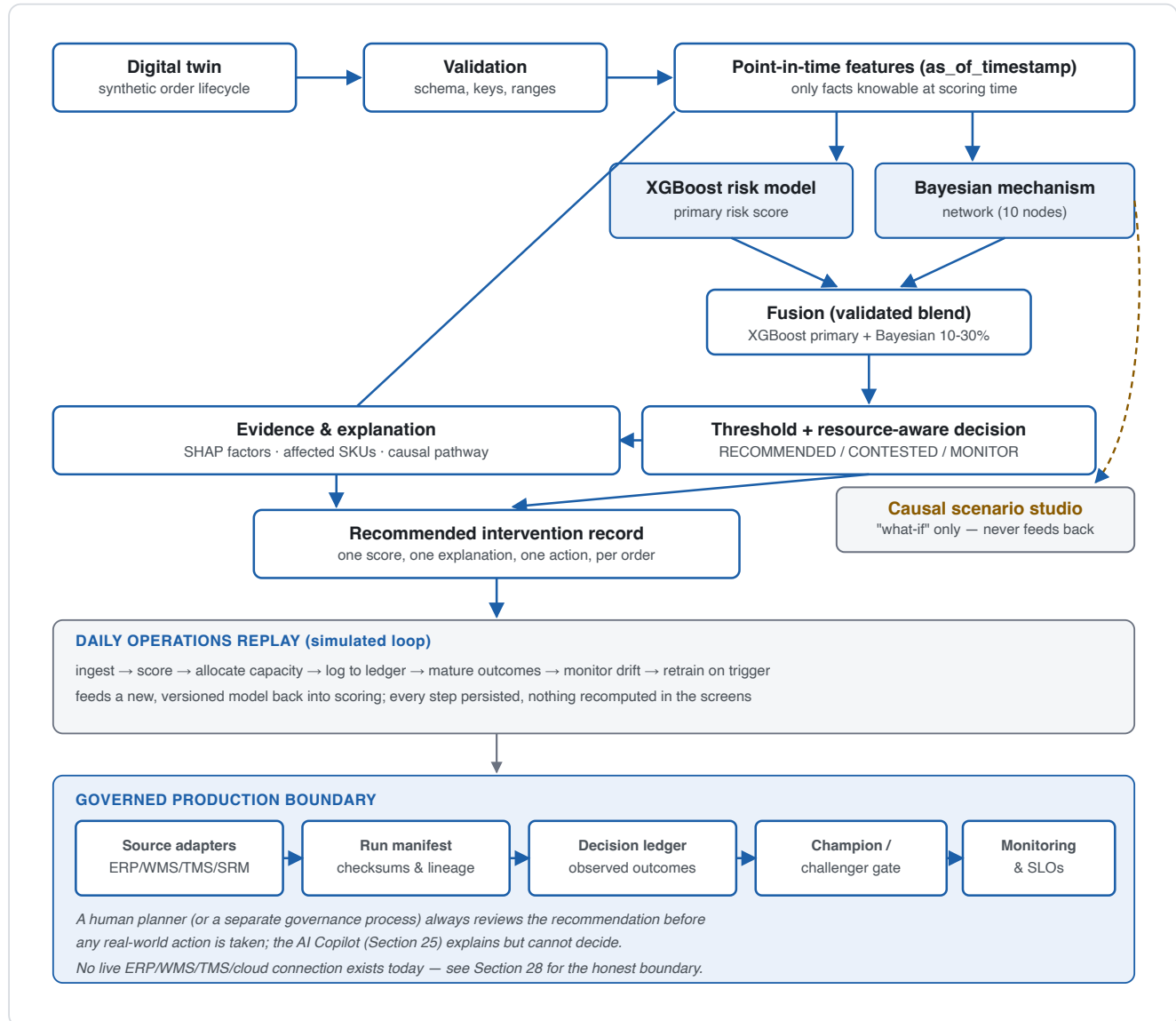


Figure 1. End-to-end flow: practice data enters at top left, a prediction and an explanation are produced, a resource-checked decision comes out the bottom, and the whole loop is replayed daily and wrapped in a governance boundary before anything would touch a real system.

3.1 Plain-language walk-through

1. **Digital twin.** A simulator generates a realistic set of orders, events, and capacity data (Section 4) because no real operational data is available to this prototype.
2. **Validation.** Before anything else runs, the data is checked for structural problems — missing columns, duplicate identifiers, impossible quantities (Section 6).
3. **Point-in-time features.** Raw data is turned into a table of measurements ("features") that are honestly knowable at the moment a prediction would be made — never using information from the future (Section 7–8).

4. **Two models score every order.** An XGBoost model (Section 10) produces the primary risk score. A 10-node Bayesian network (Section 13) produces an independent, more-interpretable risk estimate and can also run "what if we fixed X?" scenarios (Section 14).
5. **Fusion.** The two scores are combined using weights chosen on validation data only (Section 15).
6. **Decision.** The fused score is compared against a threshold; if the order is risky enough, the system checks whether the people/equipment needed for a fix are actually available and produces one of three statuses: RECOMMENDED, CONTESTED, or MONITOR (Section 16).
7. **Evidence and explanation.** In parallel, the system estimates which specific products (SKUs) on the order are affected (Section 12) and produces a plain-language explanation of the top contributing factors (Section 11).
8. **Daily operations replay.** All of the above is repeated once per simulated day across a 90-day window, with capacity allocated, outcomes recorded, drift checked, and the model retrained on a schedule (Section 19).
9. **Governance boundary.** A separate layer of production-shaped adapters, tamper-evident manifests, a decision ledger, and a champion/challenger promotion gate wraps the whole loop, so that no unvetted model version and no unverified data source could reach a real deployment without passing documented checks (Sections 20–24).

4. The practice data: a noisy digital twin

4.1 Why synthetic (made-up) data at all?

This prototype was built without access to a real company's order, warehouse, or shipping records. Rather than skip the data problem, it builds a **digital twin**: a computer simulation that behaves like a real supply chain closely enough to generate realistic orders, delays, shortages, and outcomes — with all the underlying "truth" about what actually happened kept on hand for grading the model afterward. This lets every claim in this document be checked against known ground truth, something that would be much harder with real, partially-unlabeled operational data.

4.2 Outcomes emerge — they are not preselected

A common and easy mistake when building a practice dataset is to first decide "this order will miss OTIF" and then plant matching evidence for a model to find — which teaches the model to recognize planted evidence, not real risk. This system does the opposite: for every order, it simulates each stage of the order's actual journey one at a time —

1. order capture (how long after the customer's request did the order get logged),
2. vendor readiness (does the vendor have the goods ready on time),
3. line-level allocation (is there enough inventory for every product line on the order),
4. warehouse processing (picking and packing delays),
5. transport (carrier transit time and delay),
6. customer delivery (appointment scheduling, on-site delays),

— and only *after* every stage has run, the system adds up the accumulated delay and looks at the final shipped quantity to determine whether the order was, in fact, on time and in full. Nothing about the final outcome is chosen in advance; it is the mathematical consequence of everything that happened at each stage, exactly the way it would be in a real supply chain.

4.3 The ingredients that make outcomes realistic

STABLE HETEROGENEITY

Vendors, distribution centers (DCs), lanes (shipping routes), customers, and products (SKUs) each have their own persistent traits — e.g. one vendor is simply more reliable than another, one product is scarcer, one customer reschedules appointments more often. These traits do not change from order to order.

SEASONALITY

A smooth calendar-wide curve raises or lowers general demand and congestion over time, the way retail volume rises before a holiday.

NOISE

Every stage adds random variation drawn from statistical distributions (gamma and normal curves), not a single fixed number — two otherwise-identical orders will not have identical delays.

CORRELATED SHOCKS

Randomly-timed disruption windows hit a specific vendor, DC, or lane for a period of days, with their own severity — modeling a real-world event like a factory slowdown or a weather event, rather than independent bad luck on every order.

A SCRIPTED DRIFT EPOCH

A deliberate stretch late in the simulated calendar with elevated vendor-disruption density, used later to test whether the monitoring system (Section 23) actually notices a changing world.

PARTIAL OBSERVABILITY

About 3% of vendor/warehouse/transit events never get logged at all, and even when a delay is real, the matching exception code is only recorded roughly 85–90% of the time. A real operations team never sees perfectly complete data either, so neither should this system's practice data.

BACKFILL / RECOVERY CHANCE

A shortage spotted at order capture has a documented chance of being fixed before shipment by an expedited replenishment. This means an early warning sign is a genuine *risk factor*, not a guarantee of failure — matching how early signals behave in the real world.

4.4 Real versus simulated, stated plainly

Every order, event, capacity number, and outcome in this system today is **generated by the simulator above**, not observed from a real company. The simulator is designed to be realistic (noisy, partially observed, seasonal, shock-prone) specifically so the modeling techniques demonstrated here would transfer to real data with only an adapter-layer change (Section 20) — but no claim is made that this system has been validated on, or would perform identically against, real operational data.

4.5 Ground truth is kept — but locked away from the model

The simulator secretly knows exactly which shock hit which order, exactly how much delay accumulated at each stage, and exactly which product lines were genuinely short. This "simulator truth" is written to its own separate files (`simulator_truth.csv` , `line_truth.csv` , `shocks.csv`) and is used **only** to grade the model after the fact — for example, to check whether the model's guess about which SKU is affected actually matches which SKU was truly short. The model itself never sees these files; if it did, it would be cheating by looking at the answer key instead of learning from observable evidence.

4.6 Five reserved scenarios

To guarantee a small number of instructive, reproducible situations exist in every run regardless of which random seed is used, five specific orders are always constructed the same way: a multi-cause cascade (several problems compounding on one order), two orders that end up competing for the exact same scarce recovery resource, a single-line stockout inside an otherwise-healthy multi-line order, and one genuinely unexplained miss with no recoverable evidence at all. These are discussed with real, measured numbers in Section 27.

5. The ten data tables, explained

The digital twin writes out ten tables. Row counts below are from one canonical run of 2,500 orders (seed 42) and will vary slightly with the random seed or order count, but the structure and relationships are always the same. "Grain" means what one row represents.

Table	Rows	Grain (one row =)	Important fields	Example	Why it exists
orders	2,500	one customer order	order_id , order_date , promised_delivery_date , vendor/DC/lane/customer IDs, capture_delay_hours	Order 0000001 promised for delivery on 2024-03-10, routed through vendor V006 and DC DC005 .	The central unit the whole system scores and reports on — one risk score and one decision per order.
order_lines	4,965	one product line on one order	order_line_id , sku_id , requested_qty , allocated_qty , shipped_qty , stockout_flag	Line 1 of order 0000001 requested 23 units of SKU019 ; 23 were allocated and shipped (no stockout).	OTIF's "in full" half is decided at the line level — an order can be short on just one product while every other line ships complete.
events	9,784	one timestamped lifecycle event	order_id , event_type (e.g. VENDOR_READY , SHIPPED , DELIVERED), planned_timestamp , event_timestamp , exception_code	0000001 's VENDOR_READY event was planned for 2024-03-01 00:00 and actually posted 2024-03-01 02:31.	The raw, timestamped trail every feature and every point-in-time snapshot is built from.
capacity_snapshots	1,635	one DC's capacity on one calendar day	dc_id , snapshot_date , available_capacity_units , planned_units , utilization	DC001 had 1,200 available units and 538 planned units on 2024-01-01 (45% utilization).	Lets the system know whether a distribution center is already stretched thin before recommending a warehouse-side fix.

Table	Rows	Grain (one row =)	Important fields	Example	Why it exists
vendors	12	one supplier	vendor_id , reliability_score , delay_scale_hours , contract_lead_days	Vendor V001 : reliability 0.943, typical delay scale 4.25 hours.	Persistent vendor traits drive vendor- caused delay and are the basis for "vendor reliability history" features.
dcs (distribution centers)	5	one warehouse/DC	dc_id , daily_capacity_units , capacity_variability	DC001 : 1,200 daily capacity units, 13.2% capacity variability.	Defines how much a DC can process per day and how consistent that is — the basis of DC utilization and recovery- capacity logic.
lanes	15	one shipping route from a DC to a destination region	lane_id , origin_dc_id , destination_region , planned_transit_days , transit_variability_days	Lane L001 : from DC002 to the CENTRAL region, planned 2-day transit, carrier BlueLine.	Captures how variable and how long transport takes on a given route.
customers	40	one customer account	customer_id , region , appointment_required , reschedule_trait	Customer C001 in the CENTRAL region does not require a delivery appointment and rarely reschedules (trait 0.036).	Some customers require scheduled delivery appointments, which adds its own risk of delay independent of the rest of the supply chain.

Table	Rows	Grain (one row =)	Important fields	Example	Why it exists
skus (products)	120	one product	sku_id , criticality_tier , base_unit_value , scarcity_trait , demand_volatility	SKU001 : STANDARD tier, \$22.44 unit value, low scarcity (0.025).	Product-level traits feed the affected-SKU evidence model (Section 12) and the business value of a potential miss.
shocks	19	one disruption window	shock_id , shock_type (e.g. VENDOR_DISRUPTION), entity_id , start_date , end_date , severity	SHK0001 : a vendor disruption at V001 from 2024-03-17 to 2024-04-02, severity 2.83.	The hidden, ground-truth cause of many delays — used only to grade the model, never fed to it.

5.1 Two tables that never touch the model

Alongside the ten tables above, the simulator also writes `simulator_truth.csv` and `line_truth.csv` — the answer key. These record, per order and per order line, exactly which shock caused what and exactly how much quantity was truly short. They exist only so the evaluation code (which grades the model, Sections 11–13) can check the model's guesses against reality. Every place in this codebase that reads these two files is a test or an evaluation script — never the model training or scoring code — and this separation is itself checked by automated tests, so a future code change cannot accidentally leak the answer key into the model without a test failing.

6. Data validation: catching bad data before it spreads

Before any table is used for anything, a validation step checks it for structural problems and immediately stops the entire run ("fails fast") if any check fails — rather than silently producing a model trained on broken data. The checks fall into four families:

Check family	What it verifies	Example failure it would catch
Schema	Every required table exists, is not empty, and has every column the rest of the system expects (e.g. <code>orders</code> must have <code>order_id</code> , <code>promised_delivery_date</code> , <code>vendor/DC/lane/customer IDs</code>).	A generator change accidentally drops the <code>promised_delivery_date</code> column — caught immediately with "orders missing required columns: ['promised_delivery_date']" instead of a confusing failure three steps later.
Keys	Identifier columns (like <code>order_id</code> , <code>vendor_id</code>) contain no missing values and no duplicates.	Two rows share the same <code>order_id</code> — caught as "orders.order_id must be unique" before it can silently double-count an order anywhere downstream.
References	Every foreign key actually points at a row that exists — e.g. every order's <code>vendor_id</code> must exist in the <code>vendors</code> table, every order line's <code>order_id</code> must exist in <code>orders</code> .	An order line references a SKU that was never generated — caught rather than producing a feature row with a silently blank product trait.
Quantities and timestamps	Requested quantities are positive, allocated/shipped quantities are non-negative, every order has exactly one <code>DELIVERED</code> event, and event timestamps respect logical ordering.	An order somehow has zero or two <code>DELIVERED</code> events — caught as "every order must have exactly one <code>DELIVERED</code> event" rather than silently producing an undefined OTIF outcome for that order.

Why this matters even though the data is synthetic: validation is exactly the kind of check a real ERP/WMS integration would also need (duplicate order IDs, orphaned references, impossible quantities are common real-world data-quality problems). Building and testing this discipline now means the same validation module can run unchanged against real source-system data later, simply behind the adapters described in Section 20.

7. Point-in-time scoring and data leakage

7.1 The core idea

Every prediction this system makes is tagged with an **as-of timestamp**: the exact moment in time at which the prediction is imagined to be made. A feature is only allowed to use information that had genuinely already happened by that moment. This might sound obvious, but it is one of the easiest mistakes to make by accident when building a predictive system, and it has a specific name:

Data leakage is when a model is accidentally given information from the future relative to the moment it is supposed to be making a prediction. A model trained with leaked information can look extremely accurate in testing — because it is effectively being shown the answer — and then fail badly the moment it is used for real, forward-looking predictions, where the future genuinely is not known yet.

7.2 A concrete example

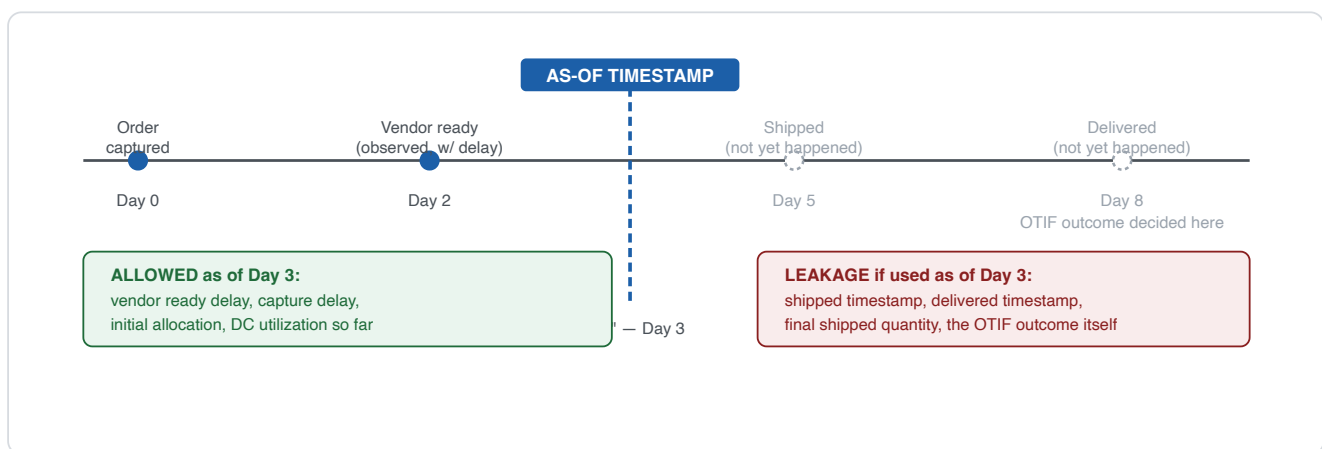


Figure 2. An order captured on Day 0 is being scored "as of" Day 3. The vendor-ready event (Day 2) already happened and may be used. The shipped and delivered events (Days 5 and 8) have not happened yet from the as-of viewpoint and must not be used — even though, at the time this document is generated, the full history for training data already exists in storage.

7.3 How the system enforces this — not just promises it

Every feature is computed strictly from events whose own recorded timestamp is at or before the as-of timestamp being scored, and from facts known at order capture (like the initial inventory check) or from slowly-changing reference data (like a customer's general appointment requirement). The same as-of contract is used in two different places: (1) when reproducing a historical snapshot for training/testing, and (2) when scoring an order live in the daily operations replay — so the exact same code path is exercised both times, rather than one "training" version and a separate, easier-to-get-wrong "production" version.

This is not just a design promise; it is checked automatically. Part of the automated test suite proves three things: (a) a feature that looks forward in time is *not* present in the simulator's raw output at all — it is derived only from already-observed operational fields — (b) that feature is not a perfect stand-in for the hidden true cause (i.e. it carries real uncertainty, not a leaked answer), and (c) changing every future event and outcome for an order cannot change an earlier as-of snapshot's row for that same order at all. If a future code change ever violated the as-of contract, these tests would fail immediately.

8. Turning raw data into features

A **feature** is a single measurable number or category that the model is allowed to look at when making a prediction — for example, "hours the vendor was late" or "how many of the last 30 days' orders from this vendor had a fault." Building good features from raw event and order data is called **feature engineering**. This system builds roughly 100 features per order, grouped as follows.

Feature group	What it captures	Example
Line aggregates	Facts rolled up from the order's individual product lines into one order-level number.	<code>worst_line_shortage_ratio</code> — the single worst shortfall across all lines on the order.
Entity traits	Stable characteristics of the vendor, DC, lane, customer, and SKUs involved.	Vendor reliability score, DC capacity variability, whether the customer requires a delivery appointment.
Observed events	What has actually posted so far, converted into a signal — never the hidden cause directly.	<code>vendor_ready_delay_hours</code> , an exception code observed at the warehouse stage.
Timing / slack	How much buffer remains between now and the promised delivery date.	<code>remaining_slack_hours</code> — if this is small or negative, there is little room left to recover from any delay.
Rolling history (30-day / 90-day / all-time)	How this vendor, DC, lane, customer, or SKU has performed recently and historically, computed only from data strictly before the as-of date.	<code>vendor_rolling_fault_rate_90d</code> — this vendor's fault-attributed miss rate over the last 90 days (see note below).
Freshness / missingness	How stale the available information is, and how many expected events have not shown up at all — itself a risk signal, since a quiet order can mean either "everything is fine" or "we simply have not heard back yet."	<code>hours_since_last_observed_event</code> , <code>missing_event_stage_count</code> .

A subtle but important detail — fault attribution. A vendor's rolling history feature counts only misses that were *actually caused by that vendor*, not every miss on orders that vendor happened to touch. If a vendor's order missed OTIF because of a warehouse problem, that miss does not count against the vendor's own reliability history. Without this distinction, a perfectly reliable vendor could look unfairly risky simply by being paired with an unreliable warehouse.

8.1 Training/scoring parity

"Parity" here means: the exact same feature-building code produces the exact same feature values whether it is run once during model training/testing, or run every day inside the operations replay when scoring a still-open order. There is no separate, simplified "fast path" used only in production that could quietly drift out of sync with the version used for training. This is proven directly later in the document (Section 20's offline/batch parity result): the same order and the same as-of snapshot, scored two different ways, produced byte-identical feature vectors and scores.

9. Why the data is split by time, not at random

To measure how well a model performs on data it has not seen, the available orders are divided into three groups: a **training** set (used to fit the model), a **validation** set (used to tune settings like the decision threshold), and a **test** set (used only at the very end, to report an honest performance number). This system splits **chronologically** — training on the earliest 60% of orders by calendar time, validating on the next 20%, and testing on the final 20% — rather than shuffling orders randomly into the three groups.

9.1 Why not a random split?

A random split would let orders from later in the calendar end up in the training set even though, in the real deployment this system imitates, the model would never have seen the future before making a prediction. A model trained partly on "future" orders relative to some of its test orders would be quietly cheating — its test-set score would look better than it would ever perform in a genuine forward-looking deployment. Splitting by calendar time instead means every test-set prediction is honestly evaluated exactly the way it would be made in practice: using only what came before.

One additional detail: orders that share the exact same as-of timestamp are always kept together on the same side of a split boundary — the split never cuts a group of identically-timestamped orders in half, which would otherwise let a handful of same-moment orders leak information across the training/test boundary.

9.2 Rolling-origin evaluation

A second, more advanced check — used specifically to evaluate the Decision Value Lab's policies (Section 17) — goes further: the calendar is divided into five equal chronological slices. The first slice is treated purely as historical "warm-up" data (there is no earlier history yet to train on, so it is excluded from evaluation entirely). Each of the remaining four slices is scored only by a model that was trained — and had its fusion weight and threshold selected — using history strictly *before* that slice. This is called **rolling-origin** evaluation: the "origin" (the training cutoff) rolls forward through the calendar, and no slice's own rows are ever allowed to influence the model that scores it. Coverage (roughly 80% of the calendar, since the first slice is excluded) is reported explicitly rather than assumed.

10. The risk model (XGBoost), in plain language

10.1 What XGBoost is, without the jargon

XGBoost is a widely-used machine learning technique that builds its prediction out of many simple decision rules ("decision trees"), each one a short chain of yes/no questions like "Is remaining slack under 12 hours? Is vendor reliability below 0.85?". No single tree is very accurate on its own. XGBoost builds hundreds of small trees one after another, where each new tree focuses specifically on correcting the mistakes the trees before it made, and adds all of their votes together into one final number. This "boosting" approach tends to perform very well on the kind of structured, tabular business data this system uses (rows of orders with columns of numbers and categories), which is why it is a standard, well-understood choice for this type of problem rather than something exotic.

10.2 Inputs and outputs

WHAT GOES IN

The roughly 100 point-in-time features for one order described in Section 8 — vendor/DC/lane/customer/SKU traits, observed event signals, timing slack, rolling history, freshness and missingness.

WHAT COMES OUT

A single number between 0 and 1 representing the model's estimated probability that this order will miss OTIF, given everything it can currently observe.

10.3 Calibration

A raw model score is not automatically a trustworthy probability — a model can be systematically overconfident or underconfident. **Calibration** is an extra adjustment step, fit on the validation set, that stretches or compresses the raw scores so that, for example, orders the model calls "70% likely to miss" really do miss roughly 70% of the time in practice. This system uses a standard calibration technique (Platt scaling) fit only on validation data, never on the test data used for final reporting.

10.4 Threshold

A risk score alone is not a decision. A **threshold** is the cutoff above which an order is considered "risky enough to act on." This system tunes its threshold to target a recall of 65% while keeping precision no lower than 30% (see the box below for what those words mean), chosen on validation data only. On the canonical single run, this produced a threshold of 0.274 on the fused (see Section 15) score.

10.5 False positives and false negatives, explained simply

False positive — the model flags an order as at-risk, but it actually ships OTIF-complete anyway. Cost: a planner's attention (and possibly scarce recovery capacity) is spent on an order that did not need it.

False negative — the model does not flag an order, but it misses OTIF anyway. Cost: a real problem goes unnoticed and unaddressed until it is too late to fix.

Precision is: of the orders the model flagged, what fraction really did miss? **Recall** is: of the orders that really did miss, what fraction did the model flag? These two normally trade off against each other — casting a wider net (higher recall) usually catches more false positives too (lower precision) — which is why the threshold above is chosen deliberately, with a floor on each.

10.6 Measured performance (5-seed benchmark, 2,500 orders per seed)

All numbers below are medians across five independent random seeds, with the observed range across those seeds in parentheses — reported as measured, not tuned to look good.

Metric	Median	Range across 5 seeds	Plain meaning
Fused PR-AUC (test)	0.705	0.535–0.763	Precision-Recall Area Under the Curve — a single number (0 to 1, higher is better) summarizing how well the score ranks orders that truly miss above orders that do not, across every possible threshold. More informative than plain accuracy when misses are the minority outcome.
Fused recall (test)	0.705	0.633–0.856	At the chosen threshold, about 70.5% of true OTIF misses are flagged in advance.
Fused precision (test)	0.614	0.509–0.635	About 61.4% of flagged orders genuinely go on to miss OTIF.
Fused Brier score	0.097	0.074–0.099	A calibration-quality score (lower is better; 0 is perfect) measuring how close predicted probabilities are to actual outcomes.
XGBoost-only PR-AUC	0.700	0.533–0.771	The XGBoost model's own accuracy before blending with the Bayesian network — very close to the fused number, since XGBoost is the dominant contributor (Section 15).

Source: `uv run otif-benchmark --seeds 1 2 3 4 5 --orders 2500`, persisted in `artifacts/benchmark.json`; see `docs/model-card.md` for the complete table.

11. Explaining a single score (SHAP)

11.1 The problem SHAP solves

XGBoost's final score is the combined result of hundreds of small decision trees — not something a person could read off directly. **SHAP** (SHapley Additive exPlanations) is a well-established statistical technique, borrowed from game theory, that answers the question "how much did each individual feature push this one order's score up or down, compared to an average order?" It does this fairly — attributing credit/blame across features in a mathematically consistent way — rather than just picking whichever feature looks biggest.

When the SHAP library is unavailable in a given environment, this system falls back to a deterministic alternative: it perturbs (temporarily changes) one feature at a time and measures how much the score moves, producing a similar factor-by-factor explanation without depending on an extra library.

11.2 Association, not causation

This is the single most important caveat in this entire document. A SHAP value tells you that a feature was *statistically associated* with a higher or lower score — never that changing that feature would *cause* the outcome to change. Every SHAP-derived explanation in this system is explicitly labeled `association_not_causation` for exactly this reason. For example, "low remaining slack contributed +0.18 to this order's risk score" does not mean that artificially adding slack would fix the order — it means the model learned that orders with low slack historically missed more often.

11.3 A worked example

Suppose an order's fused risk score is 0.81 (well above the 0.274 threshold). A SHAP breakdown might read:

Factor	Contribution to the score	Plain-language meaning
<code>vendor_ready_delay_hours</code> = 38.2	+0.21	The vendor posted its "ready" event about 38 hours later than planned — a strong upward push.
<code>allocation_ratio</code> = 0.61	+0.17	Only 61% of requested quantity was allocated at capture time — a meaningful shortfall risk.
<code>remaining_slack_hours</code> = 9.4	+0.09	Very little buffer time remains before the promised delivery date.
<code>vendor_rolling_fault_rate_90d</code> = 0.04	−0.03	This vendor's own recent fault-attributed miss rate is low — a small downward push, partly offsetting the other factors.

Reading this, a planner immediately understands *why* the model is worried — a late vendor and a real allocation shortfall, with little time buffer left — without needing to understand the underlying decision trees.

12. Which products on the order are affected

An order-level risk score alone does not tell a planner which specific product(s) on a multi-line order are actually the problem. A second, smaller model looks at each product line individually and flags it as "likely affected" using only information that was genuinely knowable at capture time: the initial available-to-promise (ATP) allocation, the inventory snapshot at the time of order, the product's own criticality tier, and the order's own point-in-time vendor-exception signal. Critically, this line-level model never looks at the hidden simulator answer key (`line_truth.csv`) while making its guess — that file is used only afterward, to grade it.

These per-line flags are also rolled up into safe order-level summary features — the worst shortage ratio across all lines, how many lines are flagged, what share of flagged lines are "critical" tier products, and how concentrated the order's quantity is — which then feed back into the order-level XGBoost model from Section 10 as additional, still point-in-time-safe features.

12.1 Measured accuracy

	Precision	Recall	Basis
Targeted line evidence (5-seed median)	0.536 (range 0.507–0.602)	0.627 (range 0.560–0.726)	5-seed benchmark, held-out test lines
Targeted line evidence (canonical seed-42 run)	0.58	0.66	Single canonical run, held-out test lines
Naive "flag every held-out line" baseline	0.09	1.00	Same held-out lines, for comparison

The naive baseline's recall is a trivial 1.00 because it flags *every* line — it "catches" every truly affected line only by also wrongly flagging almost everything else, which is why its precision (0.09) is so poor. The targeted model is roughly six times more precise than that naive baseline at broadly comparable recall — meaningfully narrowing down which product on a multi-line order actually needs attention, rather than telling a planner "maybe all of them."

13. The 10-node cause-and-effect network

13.1 What a Bayesian network is, in plain terms

A **Bayesian network** is a diagram of boxes ("nodes") connected by arrows, where each arrow represents "this can influence that." Each node also carries a table of probabilities (a **Conditional Probability Table**, or **CPT**) describing how likely it is to be "active" given the state of the nodes pointing into it. Unlike XGBoost (Section 10), which is a single opaque score, a Bayesian network's structure is fully visible and its numbers can be read node by node — which is why it is used here specifically for interpretability and for "what if" scenario analysis (Section 14), even though, on its own, it is a less accurate predictor than XGBoost (Section 13.5).

13.2 This system's specific network

Seven nodes represent operational causes that map to real supply-chain stages; two "mechanism" nodes separate *quantity* problems from *timing* problems (matching the actual definition of OTIF from Section 2, instead of collapsing everything into one flat list of causes); and one final node represents the OTIF miss itself.

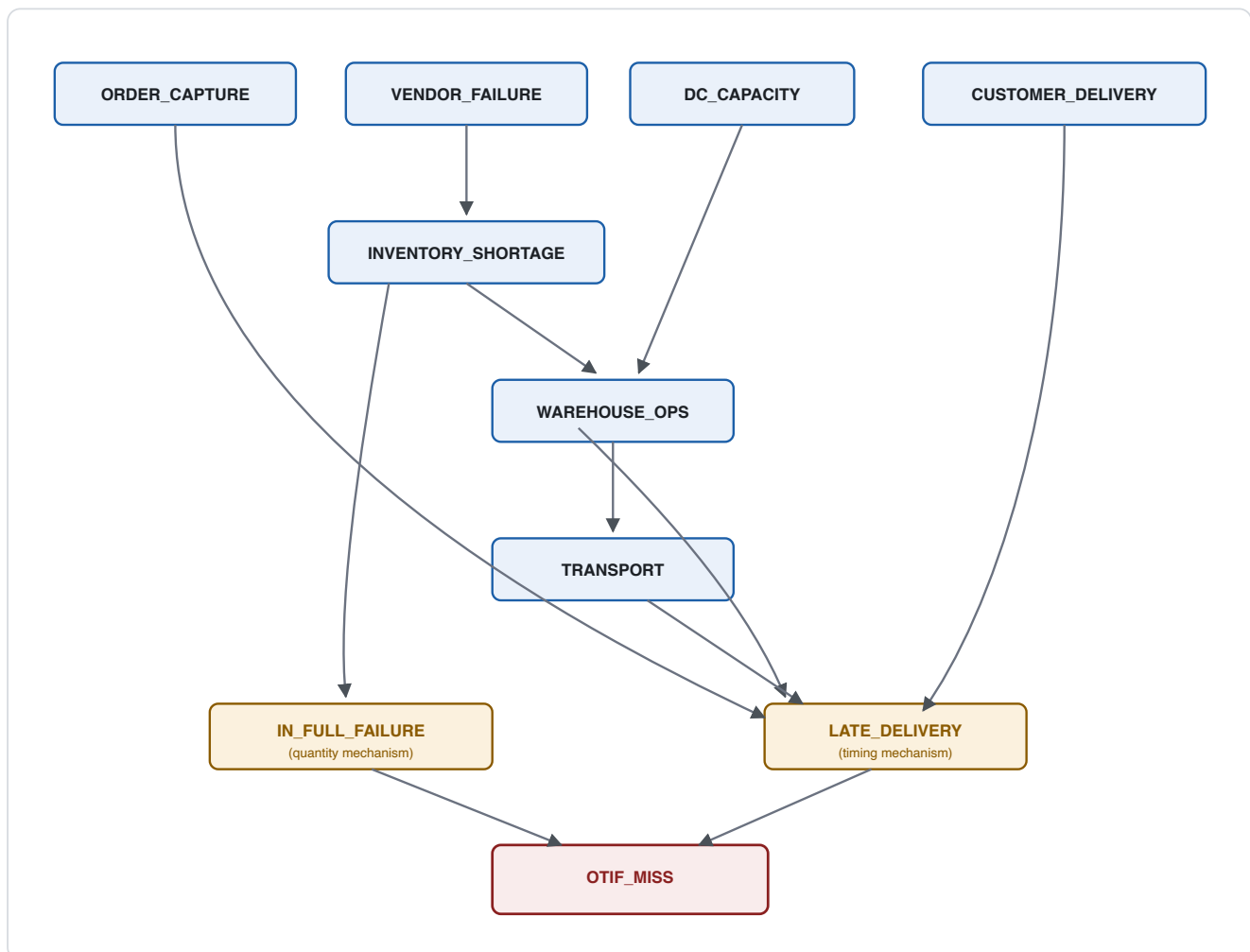


Figure 3. The 10-node mechanism network. Blue boxes are the seven operational cause nodes; amber boxes are the two mechanism nodes that separate a quantity problem from a timing problem; the red box is the final OTIF-miss outcome. A shared upstream cause (like `VENDOR_FAILURE`) can have a route to *both* mechanisms at once.

13.3 Conditional probability tables (CPTs) and inference

Each node's CPT is fit from counts observed in the training data only — for example, "of all orders where `VENDOR_FAILURE` was active and `DC_CAPACITY` was not, what fraction also had `WAREHOUSE_OPS` active?" These

counts include disruptions that were successfully absorbed without causing an OTIF miss, not just disruptions on orders that went on to miss — so the network learns the full picture, not only the worst cases. The two mechanism nodes are fit directly from each training order's own actual "was it late" / "was it short" outcome, rather than being guessed from the seven cause labels.

Inference is the process of computing "given what we've actually observed, what is the probability of OTIF_MISS (or any other node)?" This system computes exact inference — not an approximation — using a well-established open-source library (`pgmpy`) when it is available, or a from-scratch brute-force calculation over every possible combination of the network's states when it is not. Both methods have been checked, node by node, to produce numerically identical answers, so which one happens to run does not change any result.

13.4 Missing evidence is not assumed absent

A node is only treated as directly observed once its corresponding stage has actually happened as of the as-of timestamp. If, for example, an order has not yet reached the warehouse stage, `WAREHOUSE_OPS` is not simply assumed to be "false" — it is **marginalized out**: the network mathematically averages over both possibilities (active and not active), weighted by how likely each is given everything else that is known, rather than guessing. This lets an early vendor-failure signal raise OTIF risk immediately, well before the warehouse or transport stages have even occurred.

13.5 Measured performance and an honest limitation

Metric	Median	Range	Meaning
Bayesian-only PR-AUC (overall OTIF miss)	~0.475	0.346– 0.486	Meaningfully weaker on its own than XGBoost's 0.700 — expected, since this network optimizes for a transparent, fixed structure rather than raw predictive power.
<code>IN_FULL_FAILURE</code> mechanism PR-AUC	~0.417	0.312– 0.451	Against its own roughly 13–15% held-out base rate — a solid, useful signal for the quantity mechanism.
<code>LATE_DELIVERY</code> mechanism PR-AUC	~0.126	0.057– 0.149	Against its own roughly 3.8% held-out base rate — a weak predictor (around 3x lift over random, but far weaker than the quantity mechanism).

Honest limitation. Most modeled OTIF misses in this digital twin are dominated by quantity shortfall rather than timing delay, and the network's ability to predict *timing* failures specifically is genuinely weak. This is a measured property of the fitted network on this particular dataset, reported plainly rather than adjusted to look better — any timing-side pathway detail from this network should be read with lower confidence than quantity-side detail.

14. "What if we fixed this?" scenario analysis

14.1 What a structural intervention is

Beyond simply asking "given what we observe, what is the risk?", the Bayesian network from Section 13 can also answer a different, more ambitious-sounding question: "if we could force this one cause node to be fully resolved, what would the network's estimated risk become?" This is computed with a technique called the **do-operator** — mathematically different from simply pretending you had observed that node to be resolved. The do-operator temporarily replaces that node's learned probability table with a fixed value and removes the influence its own upstream causes would normally have on it, isolating the effect of that one node alone. Every scored order at risk gets one such scenario per active piece of evidence, plus one scenario combining every active fix at once.

14.2 Evidence attribution

A related but distinct diagnostic, **evidence attribution**, asks: "how much would the posterior risk change if we pretended we had never observed this one cause node at all (rather than forcing it to a fixed resolved value)?" This is computed by leaving one piece of evidence out at a time and re-running inference. It is explicitly not the same technique as SHAP (Section 11) and not a causal-effect estimate — it is a sensitivity measure for this one fixed network.

14.3 Neither of these ever feeds back into the decision

Structural intervention scenarios and evidence attribution are computed and stored purely as diagnostics for a human analyst to review. They never change the XGBoost score, never change the fused score, and never change the operational decision (Section 16). They are surfaced only in a dedicated "Causal Intelligence Studio" screen (Section 26).

14.4 A concrete worked example

On order `0002497` (introduced fully in Section 27), active evidence includes `VENDOR_FAILURE`, `INVENTORY_SHORTAGE`, and `WAREHOUSE_OPS`. The scenario table for this order reads:

Scenario	Baseline risk	After intervention	Reduction
<code>do(VENDOR_FAILURE = resolved)</code>	98.4%	98.4%	0.0 points
<code>do(INVENTORY_SHORTAGE = resolved)</code>	98.4%	9.6%	88.8 points
Combined mitigation (all active nodes)	98.4%	3.6%	94.8 points

This is a genuinely instructive, non-obvious result: fixing the "obvious" root cause (`VENDOR_FAILURE`) alone does *nothing* here, because by the time `INVENTORY_SHORTAGE` is already locked in downstream, the vendor problem has already done its damage — a real statistical phenomenon at this type of network junction, not a bug. Fixing the inventory shortage directly, however, removes almost all of the risk. Every row of this table carries the label **"Fixed-structure scenario analysis — not a proven treatment effect."**

14.5 Why "not a proven treatment effect" matters

This scenario analysis assumes the fitted network's structure and probabilities are correct and complete, and computes what they imply mathematically. It has not been validated the way a real randomized controlled trial or a real operational A/B test would validate an actual intervention. It is a rigorous, transparent "if this model of the world is right, here is what follows" statement — genuinely useful for prioritizing which lever to pull first — but it should never be presented to a decision-maker as proof that pulling that lever will produce that exact result in the real world.

15. Combining the two models (fusion)

The final risk score used everywhere downstream is a weighted blend of the XGBoost score (Section 10) and the Bayesian network's own observational risk estimate (Section 13) — **fusion**. XGBoost is deliberately the dominant contributor because it is measurably more accurate on its own (median PR-AUC 0.700 versus the Bayesian network's 0.475).

15.1 How the weight is chosen

1. Score the validation and test sets using XGBoost alone, the Bayesian network alone, and every blend in between in steps of 10% (0% Bayesian, 10% Bayesian, 20%, ... 100% Bayesian).
2. On the **validation set only**, keep any blend that (a) still catches enough true misses at the planners' realistic top-priority capacity, and (b) is within a small calibration tolerance (0.002 Brier score) of the single best blend found.
3. Among the blends left standing after step 2 — which are all practically equally accurate — prefer the one with *more* Bayesian contribution, since the Bayesian network's transparent structure is valuable for explanation even when it does not add measurable accuracy.
4. Tune the final decision threshold separately, once, for whichever blend was chosen.

15.2 Why not just force an equal weight?

Forcing, say, a fixed 50/50 blend regardless of what the data shows would either waste accuracy (if the Bayesian network is much weaker that day) or waste interpretability (if it happened to be just as accurate but under-weighted). Selecting on validation data — and never on the test data used for final reporting — means the chosen weight reflects a genuine, checked trade-off rather than an assumption. In practice, this process selects 10–30% Bayesian contribution across the 5-seed benchmark (the canonical seed-42 run selected 10%), never forcing 50/50 and never discarding the Bayesian network entirely.

The full comparison table — every blend's measured accuracy on validation and test — is always saved to disk (`fusion_comparison.csv`), regardless of which blend wins, so the choice can always be audited afterward rather than taken on faith.

16. Turning a score into a decision

16.1 Eligible threshold

An order becomes a candidate for action only once its fused risk score (Section 15) is at or above the tuned decision threshold (0.274 on the canonical run — Section 10.4). Orders below the threshold are not considered actionable at all, regardless of how interesting their factors might look.

16.2 Action mapping

Each of the seven operational cause categories (Section 13) maps to one specific, human-executable action:

Primary cause	Mapped action	Resource pool consumed
ORDER_CAPTURE	Order capture correction	Customer slot
VENDOR_FAILURE	Vendor escalation	Vendor escalation slot
INVENTORY_SHORTAGE	Inventory reallocation	DC recovery capacity
DC_CAPACITY / WAREHOUSE_OPS	Warehouse expedite	DC recovery capacity
TRANSPORT	Alternate transport	Lane alternate capacity
CUSTOMER_DELIVERY	Appointment coordination	Customer appointment slot

Resource pools are finite and shared: a distribution center can only absorb so much recovery work per day, a vendor can only field so many escalation calls, and so on (Section 16.5).

16.3 Expected-value formula (the deployed heuristic)

For the numbers shown directly in the order desk today, the estimated dollar benefit of acting on a recommended order uses a simple, transparent formula:

$$\text{estimated_avoidable_penalty} = \text{estimated_penalty_exposure} \times 60\%$$

where `estimated_penalty_exposure` is order value × penalty rate × combined risk score — how much money is realistically at stake, risk-weighted. The 60% is a fixed, documented *assumption* about how effective an average intervention is — it is a fast, transparent user-interface heuristic, not a measured number. The *measured* equivalent, which replaces this assumption with an actual simulated test, is the Decision Value Lab (Section 17).

16.4 RECOMMENDED / CONTESTED / MONITOR

- **MONITOR** — the order's fused risk score is below the eligible threshold, or its primary cause has no observable mechanism to act on. No action is proposed; the order is simply watched.
- **RECOMMENDED** — the order is above threshold, an action is mapped, and the resource pool that action needs still has capacity available today. The action is proposed for a planner to execute.
- **CONTESTED** — the order is above threshold and an action is mapped, but the needed resource pool is already fully claimed by other, higher-priority orders today. The order is marked contested and the specific competing order(s) are named, so a planner knows exactly what it is competing against rather than seeing a vague "no capacity" message.

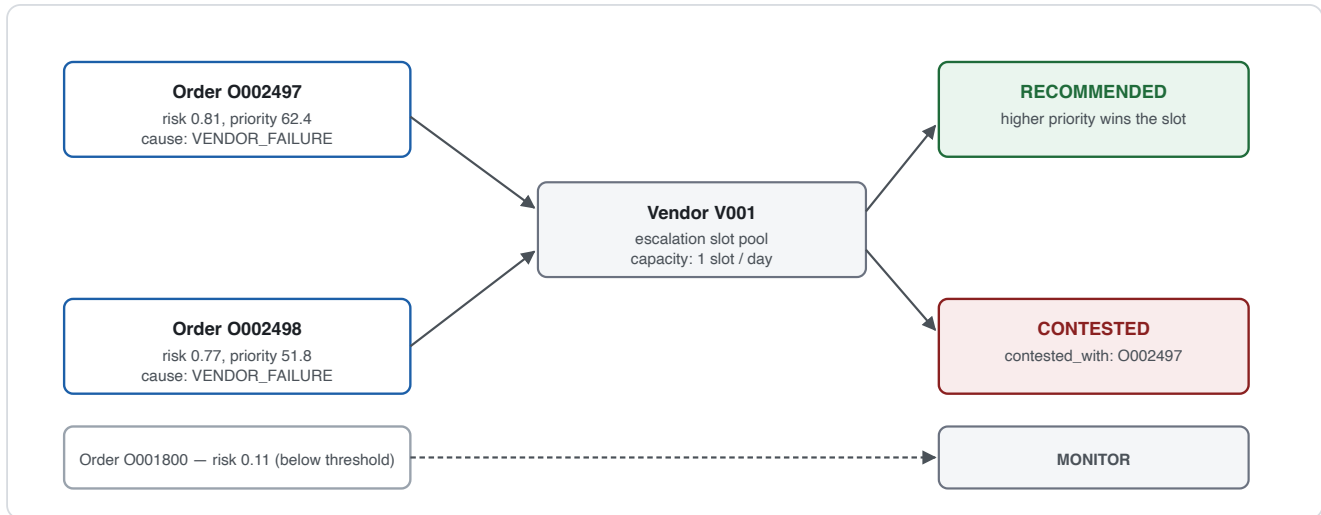


Figure 4. Two orders sharing the same vendor's single daily escalation slot: the higher-priority order is RECOMMENDED, the other is CONTESTED and told exactly which order it lost out to. A third, low-risk order never reaches the resource check at all and is simply MONITORed. This mirrors the real reserved scenario walked through fully in Section 27.

16.5 Resource demand, in one place

Distribution-center conflicts are quantity-aware (measured against real daily capacity units and a documented recovery fraction); vendor, lane, and customer conflicts are simpler count-based slot pools, since this prototype does not have an equivalent numeric recovery-capacity field for those dimensions. Every pool is allocated greedily by priority and reset each simulated day, and every "who lost to whom" is recorded — never a silent rejection.

17. The Decision Value Lab

Section 16.3 noted that the deployed "60% effectiveness" number is an assumption, not a measurement. The Decision Value Lab exists specifically to replace that assumption with an actual, falsifiable measurement — by simulating what would really happen if a given action were taken, using the digital twin's own lifecycle mechanics.

17.1 The action-response twin and common random numbers

For every combination of (random seed, order, possible action), the system draws one deterministic random number (derived from a cryptographic hash, so it is reproducible but not predictable by hand) that decides whether that specific action would have succeeded for that specific order. The key idea is **common random numbers**: every candidate policy being compared sees the *exact same* drawn outcome for the exact same order/action pair. This means differences measured between policies come from genuinely different decisions, not from different luck — a standard, rigorous simulation technique for making fair comparisons.

Six actions are modeled (matching Section 16.2's mapping, with warehouse expedite covering both DC-capacity and warehouse-ops causes). A successful action reduces only the specific delay or shortfall it targets; a failed, mismatched attempt can occasionally make the targeted problem slightly worse, reflecting genuine operational risk. One documented limitation: because this twin's shipping and transit timing depends only on the order date (never on how quickly the order was captured), the "order capture correction" action can never mechanically change a delivered timestamp or quantity in this simulation — reported honestly as a structural no-op, not hidden.

17.2 Eight policies compared

Policy	What it does
NO_ACTION	Never intervenes — the floor.
RANDOM_AT_CAPACITY	Accepts eligible orders in random order until capacity runs out.
HIGHEST_RISK_AT_CAPACITY	Accepts the highest-risk-score orders first.
HIGHEST_FINANCIAL_AT_CAPACITY	Accepts the highest-dollar-exposure orders first.
STRONGEST_SIGNAL_HEURISTIC	Accepts orders with the strongest single observed evidence signal first.
SINGLE_CAUSE_PRIORITY_BASELINE	The existing deployed risk×tier×value ranking (Section 16), with one fixed action per order's primary cause.
CURRENT_POLICY (value-aware)	Considers <i>every</i> point-in-time-feasible action per order (not just the one primary-cause action) and ranks order-action pairs by expected avoided penalty per unit of scarce resource consumed.
ORACLE_EVALUATION_ONLY	An unattainable, evaluation-only ceiling that may pick any action with hindsight — used solely to compute regret, never a deployable recommendation.

17.3 The value-aware formula

$$\begin{aligned} \text{expected_benefit} &= \text{estimated_penalty_exposure} \times \text{structural_reduction} \times \text{execution_feasibility} \\ \text{value_density} &= \text{expected_benefit} / \text{normalized_resource_fraction} \end{aligned}$$

structural_reduction comes from a persisted Bayesian intervention scenario (Section 14) when one exists for that cause, or otherwise falls back to the same 60% deployed-UI assumption from Section 16.3, scaled down if the signal is only a secondary, corroborating one. **execution_feasibility** is a fixed weighted mix of remaining timing slack, an

action-specific resource trait (e.g. vendor reliability for a vendor escalation), and how much of the Bayesian network's evidence is actually observed for this order. `normalized_resource_fraction` divides the action's resource cost by that pool's standard daily capacity, so the same ranking applies no matter how scarce capacity is that day.

17.4 Capacity scenarios: 25%, 50%, 100%

At this twin's generously-sized default (100%) daily capacity, resource pools are rarely actually contested, so a smarter *ranking* has very little to improve — most policies, including plain random selection, simply accept every eligible order. To ask the right business question (what happens when capacity is genuinely scarce), every policy is re-run at three capacity levels applied uniformly to every resource pool: 25%, 50% (the **primary headline** scenario every acceptance gate below is measured against), and 100% (kept only as a diagnostic).

17.5 Stage 1 headline result — measured, not asserted

Policy	25% capacity	50% capacity (primary)	100% capacity (diagnostic)
<code>NO_ACTION</code>	0.00	0.00	0.00
<code>RANDOM_AT_CAPACITY</code>	8.20	9.17	15.09
<code>HIGHEST_RISK_AT_CAPACITY</code>	8.06	9.17	14.96
<code>SINGLE_CAUSE_PRIORITY_BASELINE</code>	8.06	9.174	15.12
<code>CURRENT_POLICY</code> (value-aware, with Bayesian term)	8.35	9.964	17.02
<i>Value-aware policy without the Bayesian term (ablation)</i>	—	11.3445	—
<code>ORACLE_EVALUATION_ONLY</code> (ceiling, never deployable)	56.45	74.45	119.76

Units: median avoided penalty per normalized scarce-resource unit, across 5 fixed random seeds, 2,500 orders per seed. Source: `artifacts/policy_benchmark.json`.

At the primary 50%-capacity headline, the deployed value-aware policy (with its Bayesian term) beats every deployable baseline — 9.964 versus the shared baseline median of 9.174, a **+8.6%** improvement — winning 4 of 5 individually-tested random seeds against each baseline (only seed 4 is a loss, reported honestly, not adjusted away). The acceptance gate for this result is measured and recorded as **passed**.

Honest finding: the Bayesian term does not currently help this specific formula. An ablation test — the identical value-aware formula, with vs. without its Bayesian structural-reduction term — shows the *simpler* version (leading-signal fallback only, no Bayesian input) actually scores **higher**: a median of 11.3445 versus 9.964, winning in all 5 tested seeds. This does not change which policy is deployed (the value-aware policy is defined to include the Bayesian term and still clears its own acceptance gate against the older baselines), but it is reported plainly as an open question for future work, and it is the exact finding the governance walkthrough in Section 24 uses to demonstrate a realistic "hold" decision.

17.6 What the oracle is (and is not)

The oracle policy is allowed to pick, with full hindsight, whichever action would have produced the best outcome for each order — something no real deployable policy could ever know in advance. It exists purely to compute **regret** (how far short

of the best-possible outcome a real policy falls) and is never presented as something the system could actually deploy.

18. Exploration: keeping some capacity for learning

If the value-aware policy (Section 17) always assigned every scarce resource slot strictly by its own ranking, the system would only ever learn from the orders it already believes are the best choice — and could never discover whether a slightly lower-ranked candidate would actually have performed just as well or better. To avoid this, roughly **10% of capacity** is deliberately reserved for **exploration**: near-equally-ranked eligible orders are given a real, logged chance of being selected even when they are not the single top-ranked candidate.

18.1 How assignment probability is computed

For resource pools made of whole, discrete slots (vendor escalation calls, customer appointment slots), the explore share is split using seeded, weighted random rounding, so that the pool's full available slot count is always used and every candidate's exact chance of being selected is logged precisely. For continuous pools (distribution-center or lane capacity, which are divided in fractional units rather than whole slots), an exact per-order probability has no clean mathematical form, since order sizes differ — those rows honestly report their assignment probability as unavailable, together with a separately labeled reservation ratio, rather than presenting a misleading precise-looking number.

18.2 Why this matters for future learning

Exploration is what would let a future, real deployment of this system responsibly learn whether its own ranking is actually correct, rather than only ever confirming its own beliefs. In production terms: without occasionally trying a "second-best" option and observing what happens, a decision system can become confidently wrong forever. This 10% carve-out is a deliberate, logged, and reproducible way of avoiding that trap — while still deploying the top-ranked decision to the overwhelming majority of eligible orders.

19. A day in the life: the operations replay

To see how this system would behave over time — not just as a single snapshot — a **daily operations replay** simulates running the whole pipeline once per simulated calendar day across a 90-day window.

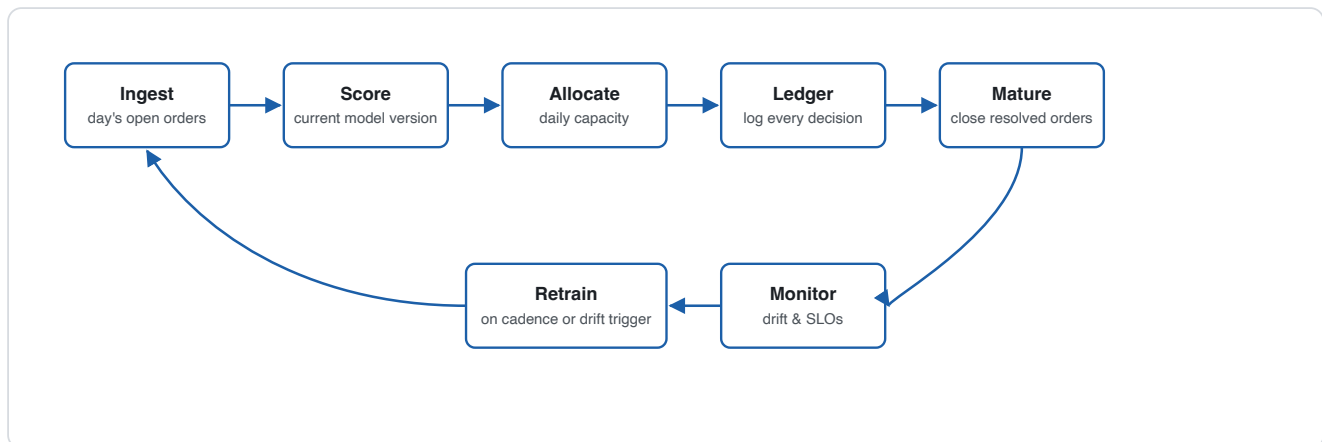


Figure 5. One simulated day's loop: still-open orders are scored by the currently active model, capacity is allocated, decisions are logged, resolved orders' true outcomes are recorded, drift is checked, and a new model version is trained if a schedule or a drift trigger fires — then the loop repeats the next day using whichever model is currently active (Section 24 decides whether a newly trained version is ever actually promoted).

19.1 What each step means

1. **Ingest.** The set of orders still open (not yet delivered/closed) as of that simulated day is assembled.
2. **Score.** Every open order gets a fresh risk score, explanation, and decision using the model version currently marked "active" (Section 24) — using the exact same point-in-time feature code as training (Section 8.1).
3. **Allocate.** Today's resource capacity (Section 16.5) is allocated across eligible orders by priority; contested orders are recorded with who they lost to.
4. **Ledger.** Every decision — feasible actions, chosen action, rejected actions, resource state before/after, execution status — is appended to a durable, idempotent ledger (Section 22).
5. **Mature.** Orders that reach their actual delivery date in the simulated calendar are closed and their real outcome (and true cause) is recorded.
6. **Monitor.** The system checks for drift — is the model's accuracy slipping, are score distributions shifting, is data going missing more than expected (Section 23)?
7. **Retrain.** On a fixed schedule, or immediately if a drift trigger fires, a new model version is trained on the freshest available history and evaluated — but, critically, it is not automatically put into service (Section 24 explains why).

19.2 What one canonical 90-day replay actually produced

Across the 90 simulated days, 9 model versions were trained in total (the initial model plus 8 retrains), 6,226 individual decisions were logged, and every one of the governance checks described in Sections 20–24 ran against real, persisted output — not a mock-up.

20. Production-shaped adapters and service contracts

Even though this prototype does not connect to any real company system today (Section 4.4), it is built as if it eventually would — through a clean, typed boundary rather than code that directly reaches into raw files everywhere.

20.1 The four source adapters

Four typed adapters stand in for the four kinds of systems a real deployment would read from: **ERP** (Enterprise Resource Planning — order and customer records), **WMS** (Warehouse Management System — inventory and warehouse events), **TMS** (Transportation Management System — carrier and transit events), and **SRM** (Supplier Relationship Management — vendor records). Today's implementations simply read from local CSV files, but they honor the exact same point-in-time contract from Section 7: any event that has not genuinely occurred yet as of the requested snapshot is redacted (not silently dropped, but explicitly marked "not yet available"), matching the same rule enforced in feature engineering. Swapping these adapters for real API/database connections later would not require changing anything upstream of them.

20.2 Service contracts

A **ScoreRequest** is a small, well-defined package containing an as-of timestamp, an order identifier, a snapshot identifier, and an idempotency key. A **ScoreResponse** returns the model/policy version, the manifest content identifier, the risk score, the threshold, a confidence label, the explanation, and the decision status. **Idempotency** means: if the exact same request is sent twice (e.g. due to a network retry), the system recognizes it as the same request and never writes a duplicate decision — it simply returns (or re-confirms) the original result.

20.3 Offline/batch parity — proven, not assumed

Measured result: the same 15 orders, scored at the same as-of snapshot, once directly against the in-memory dataset and once through the full adapter/service boundary (reading data that had been round-tripped through CSV files), produced **byte-identical feature vectors and scores** — `parity_check.json` reports `passed: true` with zero mismatched orders. This proves there is no hidden "shortcut" version of feature-building used only in one code path; both paths are provably the same computation.

21. Manifests, lineage, and tamper detection

Every run of the pipeline writes a **run manifest** — a structured record of exactly how that run was produced: the exact code version (git commit hash) and whether the working copy had uncommitted changes, the versions of every key software dependency, the configuration and schema versions used, fingerprints (row counts, schema, date ranges, content hashes) of every input table, a hash of the feature schema, the training/validation/test date windows, and a SHA-256 checksum of every other file the run produced.

21.1 Two kinds of identifiers

A **content ID** is a deterministic fingerprint that deliberately excludes anything that would differ between two otherwise-identical runs (timestamps, the run's output folder name, an internal run-instance ID). Two runs with the same random seed, configuration, code, and model-facing data will always produce the *same* content ID, even if run on different days or in different folders — proving reproducibility. Separately, a **run-instance ID** and timestamp record exactly when and where that particular run happened, for audit purposes.

21.2 Measured lineage numbers

Run	Artifacts checksummed	Verification result
Canonical single pipeline run (seed 42, 2,500 orders)	29 files	verified: true
90-day operations replay (seed 42, 2,500 orders)	121 files	verified: true

21.3 A simple tamper example

Suppose someone opened one of the persisted scored-order files after a manifest was written and manually edited a single risk score — say, changing a value to make an order look safer than the model actually said. Re-running the verification step recomputes the SHA-256 checksum of that file and compares it against the one recorded in the manifest. Because even a single-character change to a file produces a completely different checksum, the mismatch is caught immediately: verification flips from **true** to **false** and the exact file that no longer matches is named explicitly — there is no way to silently substitute a different file without it being detected.

22. The decision ledger and observed outcomes

Every decision made during the 90-day operations replay (Section 19) — one per open order, per simulated day — is written to an append/update **decision ledger**: a durable record of the feasible actions considered, the action actually chosen, resource availability before and after, the planner disposition, and execution status. Writes are idempotent by a decision key, so retrying an identical write never creates a duplicate row.

22.1 Measured numbers

TOTAL DECISIONS LOGGED	RECONCILED AGAINST A MATURED (CLOSED) OUTCOME	STILL OPEN AT REPLAY END
6,226	5,923	303
ACCEPTED COHORT MISS RATE	MONITORED COHORT MISS RATE	MEAN REALIZED PENALTY — ACCEPTED
57.96% (n = 716)	10.08% (n = 5,206)	\$113.54
MEAN REALIZED PENALTY — MONITORED		
\$13.33		

22.2 Why this is observational, not causal — and why the gap makes sense

This report is explicitly labeled `observational_not_causal`. The much higher miss rate among "accepted" orders does not mean accepting an order *caused* it to be more likely to miss — it reflects *which* orders the system chose to prioritize in the first place: correctly, the higher-risk ones. In this replay, an order's actual outcome is closed out against its pre-generated result regardless of what decision was made about it (the replay does not yet feed the chosen action back into a re-simulated lifecycle) — so this comparison is describing the policy's targeting, not measuring a treatment effect. The Decision Value Lab (Section 17) remains the only part of this system that produces a causally-interpretable, measured intervention value, and it is kept deliberately separate from this ledger.

A further breakdown by specific action taken (e.g. inventory reallocation was chosen for 431 matured decisions, with a 70.3% miss rate among them) shows the same pattern: the hardest, most at-risk orders were the ones that received a given action, so a high miss rate among "treated" orders is expected and does not indicate the action failed. Cohorts and action types with too few observations (below a minimum-sample guard) have their rate withheld rather than reported on a misleadingly small sample — e.g. the single `REJECTED` -cohort order in this replay.

23. Monitoring, drift, and service targets

23.1 What "drift" means

Drift is when the statistical patterns a model was trained on stop matching what is currently happening — for example, if vendors genuinely become less reliable over time, a model trained on older, better behavior will gradually give worse advice without any code being "broken." Left undetected, drift is one of the most common ways a working model quietly becomes a bad one. This system checks for several kinds of drift on every simulated day:

- **PSI** (Population Stability Index) — a standard statistical measure of how much the distribution of feature values has shifted compared to training data.
- **Score-distribution shift** — whether the overall spread of risk scores being produced has moved meaningfully.
- **Missingness change** — whether more data is silently failing to arrive than usual.
- **Recent OTIF-rate change** — whether the real, recently-observed miss rate has moved away from what the model expects.

23.2 Service Level Objectives (SLOs) — pass/fail, reported honestly

Metric	Target	Measured (90-day replay)	Result
Rolling realized PR-AUC	≥ 0.55	0.386	FAIL
Data-quality completeness/uniqueness/referential health	zero contract failures	zero contract failures	PASS
Feature freshness cadence	1 calendar day (structural)	1 calendar day	PASS
Measured local scoring runtime	informational only	mean 1.48 s/day	INFO
Measured local retrain runtime	informational only	mean 3.35 s (across 8 retrains)	INFO

What the failed SLO means, honestly. The rolling realized PR-AUC target (0.55) was missed at 0.386 during this replay. This is the direct, reported cost of the promotion gate (Section 24) correctly refusing to promote any of the 8 retrained model versions during this 90-day window, because every one of them measured worse than the original, still-strong champion on its own held-out test data. In other words: the system kept the safer, better model active rather than risk swapping in a noisier one, and the honestly-reported price of that caution was a stale model whose realized accuracy slowly slipped below target. This is reported as a genuine limitation of this specific 90-day run, not smoothed over — a longer replay window, or more matured outcomes per retrain, would likely narrow the noise that caused every retrain to look worse than it perhaps really was.

Runtime numbers are measured on one local development machine only and are explicitly not a production-latency claim.

24. Champion/challenger governance

A newly trained model (or a newly proposed decision policy) is called a **challenger**. The model or policy currently in active service is the **champion**. This system never swaps a challenger into service automatically — a documented **promotion gate** compares the challenger against the champion on several fixed, explicit metrics and tolerances first.

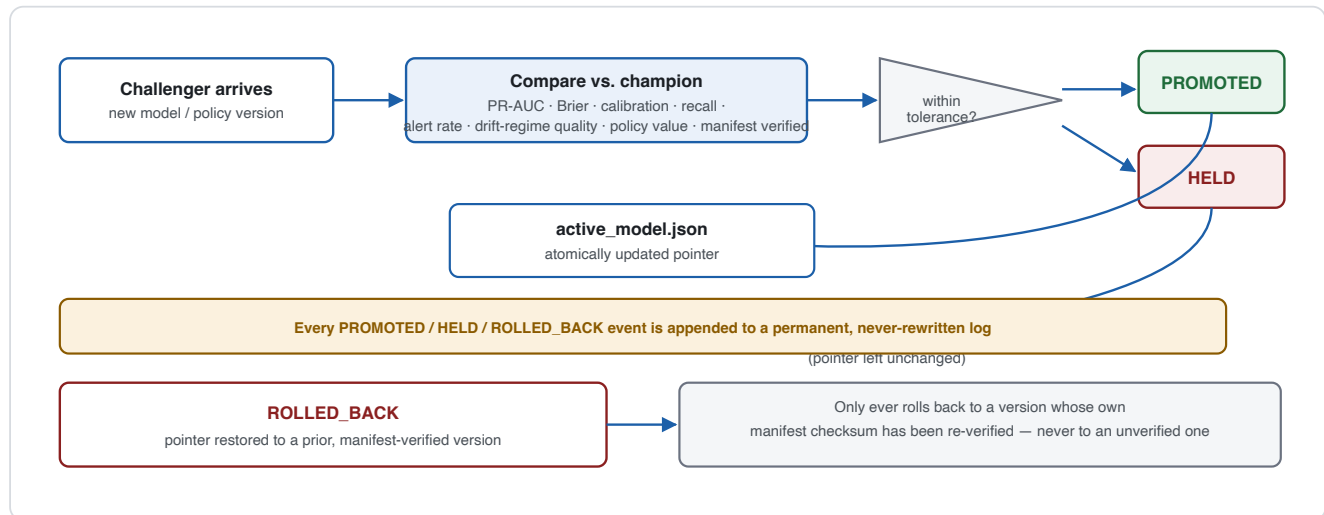


Figure 6. A challenger is only promoted if it clears every tolerance check. Every outcome — promoted, held, or rolled back — is appended to a permanent lifecycle log, and a rollback target must itself be a manifest-verified version.

24.1 The real 90-day retrain lifecycle

Across the canonical 90-day replay, 9 model versions were trained (1 initial + 8 retrains triggered by schedule or drift).

Every one of the 8 real retrain challengers was held. Each retrain's own held-out test PR-AUC — necessarily measured on a small, recent slice of matured history this early in the replay — regressed well beyond the allowed tolerance compared to the strong initial champion (for example, version 9 measured a PR-AUC of 0.343 versus version 1's 0.717). The gate correctly kept the safer, better-measured champion active for the entire replay rather than chasing noisy short-window retrains — the honest cost of this caution is the failed rolling PR-AUC SLO from Section 23.2.

24.2 The demo governance-lifecycle scenario

To demonstrate all three possible outcomes (promoted, held, and rolled back) without waiting for a real retrain to happen to produce one of each, a separate, clearly labeled *demo lifecycle scenario* is built entirely from numbers already measured in Section 17's Decision Value Lab benchmark — nothing here is invented for the demonstration.

Step	Challenger	Compared against	Policy value (50% capacity)	Result
1	Value-aware action policy <i>without</i> the Bayesian term	SINGLE_CAUSE_PRIORITY_BASELINE (9.174)	11.3445	PROMOTED
2	Value-aware action policy <i>with</i> the Bayesian term (CURRENT_POLICY)	the now-active 11.3445 champion from step 1	9.964 (-12.2%, beyond the 5% tolerance)	HELD
3	—	—	—	ROLLED_BACK to v1 , a real, manifest-verified version

The recorded reason for step 2's hold is the literal gate output: *"policy value at 50% capacity regressed 11.3445 → 9.9640 (floor 10.7773, tolerance fraction 0.05)." This scenario is explicitly, permanently labeled as built from Stage 1's own measured benchmark numbers, and is never confused with the real per-day retrain lifecycle in Section 24.1 — they are two separate, separately-logged demonstrations of the same gate logic.*

Reading this correctly: the value-aware policy *without* the Bayesian term is the one that gets promoted here, precisely because Section 17.5 measured that the Bayesian term currently makes this specific formula perform worse, not better. This is the gate working exactly as intended — rewarding the measured stronger option rather than assuming the more sophisticated-sounding one must be better.

25. The AI Copilot

A read-only explanation and drafting layer sits on top of everything described so far. It is built specifically so that it **cannot** change a score, a threshold, a RECOMMENDED/CONTESTED/MONITOR status, a resource allocation, or a model/policy promotion — it can only explain what already happened, answer a fixed list of questions, and draft text for a human planner to copy and send manually.

25.1 Evidence packets and citations

Ordinary computer code (not the language model) first builds a small, tightly-controlled **evidence packet** for one order or one fixed portfolio question: its identity and status, model/policy/manifest versions, the XGBoost/Bayesian/fused risk with the operating threshold, the top SHAP factors (labeled `association_not_causation`), the Bayesian mechanism route and its best scenario (labeled a scenario, not a proven effect), affected SKUs, observed and missing lifecycle events, resource contention, and business impact. Every single fact in the packet is given a stable identifier (like `risk.combined` or `sku.SKU0042`), and every claim the Copilot makes must cite one of these identifiers. Secrets, file paths, and the hidden simulator ground truth are never placed in the packet at all — there is no way for the Copilot to see, let alone repeat, information it should not have.

25.2 Two interchangeable answer providers, one schema

A fully offline, deterministic **fallback** mode builds the exact same structured response using plain code — no network call, no API key, nothing random. This is the system's default and always-available mode. An optional **live** mode can call OpenAI's API (default model `gpt-5-mini`, a small, cost-efficient current-generation model) when an API key is configured, using a strict, enforced output format so the response always matches the same schema either way. **The OpenAI connection is entirely optional; the deterministic fallback is authoritative** — any network error, timeout, or validation failure automatically and silently falls back to the deterministic answer rather than ever showing a blank page or an unchecked response. A real live smoke test against the actual OpenAI API using `gpt-5-mini` has been run successfully — this demonstrates the live path works when configured, but the system is designed and evaluated so it never depends on it.

25.3 What it can and cannot do

Can: explain a score and its factors in plain language; answer a fixed catalog of order-level and portfolio-level questions; draft a supplier/customer escalation message for a human to review and send; cite the exact fact behind every claim; say "unknown" when evidence is missing.

Cannot: change any score, threshold, decision status, resource allocation, or model/policy promotion; run arbitrary database queries or code; see secrets, raw file paths, or the hidden simulator ground truth; send a message on its own — every draft is copy-only.

25.4 Validation and audit

A separate validation step checks every response before it is shown: any citation pointing to a fact that does not exist in the evidence packet is rejected, every factual claim must have at least one citation, the response must explicitly confirm the persisted decision is unchanged, and non-finite numbers, oversized output, and unsupported HTML/links are stripped or rejected. Any failure here silently falls back to the deterministic response. Every single request — live or fallback — is written to a permanent audit log with a request ID, timestamp, which order or portfolio question was asked, the provider/model/mode actually used, a hash of the evidence packet (not its contents), latency, and which facts were cited — but never the API key or the full response text.

Because no human-labeled "correct explanation" dataset exists for this prototype, the Copilot's evaluation harness checks only mechanical properties — citation validity, whether the persisted decision was preserved, section completeness, fallback success rate, and latency/token usage — across a fixed set of representative order types. It makes no claim about explanation quality, factual accuracy beyond citation checking, or human preference.

26. The nine screens of the control tower

Everything described in this document is viewable through a nine-screen interactive application (built with [Streamlit](#), a Python tool for building simple data-driven web interfaces). Every screen loads only already-persisted files — nothing is recomputed live inside the interface.

Screen	What it shows
Order lookup	Search any single order: fused risk score, decision status, priority, penalty exposure, a structured narrative, the mechanism route, the affected-SKU table, any contention warning, and a planner feedback form.
Ranked portfolio	Every scored order ranked by priority, for a planner scanning the whole open-order book at once.
Hotspots + impact	Rollups by vendor, DC, lane, customer, order type, and SKU — where risk and dollar exposure are concentrated.
Operations	The daily operations replay timeline: open-order counts, drift warnings, training windows, thresholds, and fusion weights over the 90 simulated days.
Model health	The 5-seed benchmark median/range table, every acceptance gate with a pass/fail flag, and the full fusion-weight comparison table.
Causal intelligence	The 10-node network diagram for one order with active evidence highlighted, mechanism-level risk gauges, the evidence-attribution and intervention-scenario tables, and diagnostic metrics.
Policy value	The Decision Value Lab: a capacity-scenario selector, the per-policy avoided-penalty/precision/coverage/regret table, paired seed win/tie/loss counts, the value-density formula explained in place, and the honest Bayesian ablation result.
Governance	The manifest trust card, the champion/challenger lifecycle timeline with reasons, a metric-delta table, the decision ledger and observational cohorts, monitoring/SLO cards, and offline/batch parity status.
AI Copilot	An Order Copilot tab (evidence-packet preview, question selector, chat history, citation badges, a copy-only draft box) and a Portfolio Copilot tab (the fixed question catalog), plus a Copilot health card.

27. One order, start to finish

To make every preceding section concrete, this section follows one real, reserved scenario from the canonical seed-42 run through the entire system, end to end.

27.1 The setup

Two orders, 0002497 and 0002498, share the same vendor (V001), the same distribution center (DC001), and the same order date. Vendor V001 is under a forced disruption window (Section 4.3's "correlated shocks") during this period.

27.2 Data and events

Both orders' events show a late VENDOR_READY posting, an elevated DC utilization snapshot, and (for at least one of the two) a warehouse exception code. Their order lines show reduced initial allocation relative to what was requested — the observable, honestly knowable-at-capture signals from Section 8, not the hidden shock identifier itself.

27.3 Risk score

XGBoost (Section 10) raises both orders' fused risk scores well above the 0.274 threshold. SHAP (Section 11) surfaces vendor_ready_delay_hours and allocation_ratio as the leading observable factors for both.

27.4 Explanation

The retrospective root-cause derivation (used only for grading and for the causal network, never as a live feature) assigns both orders a primary cause of VENDOR_FAILURE, with INVENTORY_SHORTAGE, DC_CAPACITY, and WAREHOUSE_OPS as matched secondary causes.

27.5 The Bayesian mechanism view

For 0002497, the 10-node network (Section 13) shows active evidence for VENDOR_FAILURE, INVENTORY_SHORTAGE, and WAREHOUSE_OPS, feeding routes into *both* mechanisms: $P(\text{IN_FULL_FAILURE}) = 99.5\%$ versus $P(\text{LATE_DELIVERY}) = 9.6\%$ — this order is overwhelmingly a quantity problem, not a timing one, exactly the distinction Section 2 said a single blended score could not make. The structural scenario table (Section 14) shows fixing VENDOR_FAILURE alone changes nothing (0.0 points), while fixing INVENTORY_SHORTAGE removes 88.8 points of risk — a genuinely instructive, non-obvious result about where the real leverage is.

27.6 Action, contention, and the ledger

Both orders clear the eligible threshold and are mapped to the same action (vendor escalation), which draws on the same single-slot V001 escalation pool. The greedy priority allocator recommends the higher-priority order and marks the other contested, exactly as illustrated in Figure 4:

Order	Decision status	Detail
0002497	RECOMMENDED	Higher priority score; claims the single available vendor-escalation slot.
0002498	CONTESTED	Same eligible action, same resource pool, lower priority; contested_with names 0002497 explicitly.

27.7 Outcome and governance

Both decisions are written to the decision ledger (Section 22) with their resource state before/after and execution status. When each order matures (its actual delivery is recorded in the simulated calendar), the ledger reconciles the decision against the real outcome — feeding the observational cohort report — and, separately, the Decision Value Lab (Section 17) can replay every feasible action for an order like this one through the twin's own lifecycle mechanics to measure,

rather than assume, what a given intervention would actually have been worth. Nothing about this specific order's outcome retroactively changes the model or the threshold; any future change to either would have to pass through the champion/challenger promotion gate described in Section 24.

Why this one story matters: it touches almost every mechanism in this document at once — realistic noisy data, point-in-time-safe features, a risk score with a plain-language explanation, a mechanism-level cause breakdown, an honestly-labeled "what if we fixed it" scenario, a resource-aware decision with named contention, and a durable, auditable record of what happened next. A related reserved order (0002496) shows the same vendor/DC driving a full cause-to-cause cascade; another (0002499) shows a single stranded product line inside an otherwise-healthy multi-line order; and a final one (0002500) is a genuine, unexplained miss with no corroborating evidence anywhere — included deliberately so the system's honesty about its own blind spots (Section 29) is never just a claim on paper.

28. What is real, simulated, and not yet built

This section exists so no reader is left guessing which parts of this document describe a working, tested piece of software versus a deliberate stand-in for something a real deployment would eventually need.

Component	Status	Detail
Data generation, validation, features, XGBoost, Bayesian network, fusion, decisions, explanations	FULLY WORKING	Real, runnable code with an automated test suite; measured on synthetic data as reported throughout this document.
Decision Value Lab (action-response twin, 8-policy evaluation)	FULLY WORKING	Real code and real measured numbers — but evaluated entirely against the twin's own simulated potential outcomes, never a real-world experiment.
Daily operations replay, drift detection, retraining	FULLY WORKING	A genuine 90-day simulated replay with real persisted artifacts, not a mock-up.
Manifests, checksums, decision ledger, champion/challenger gate, monitoring/SLOs	FULLY WORKING	Real, tested governance machinery, running against real (if simulated) data.
Source adapters (ERP/WMS/TMS/SRM)	PRODUCTION-SHAPED, NOT PRODUCTION-CONNECTED	Typed interfaces with real local-CSV implementations proving the contract and offline/batch parity — deliberately not wired to any real company system, database, or message broker.
AI Copilot deterministic fallback	FULLY WORKING	Real, offline, no network dependency.
AI Copilot live OpenAI mode	OPTIONAL, VERIFIED ONCE LIVE	A real live smoke test against the OpenAI API with <code>gpt-5-mini</code> has passed, but this path is optional by design and the deterministic fallback remains authoritative — the system is built and evaluated to never depend on live network access succeeding.
Real-world causal effect of any recommended action	NOT DEMONSTRATED	Every "effect," "reduction," or "value" figure in this document is either a statistical association or a measurement made entirely inside the simulated twin — never an observed real-world experiment.
Production deployment (cloud hosting, live databases, message brokers, a real warehouse/ERP connection)	NOT BUILT	Deliberately out of scope for this prototype.

29. Results and limitations, in one place

29.1 Headline measured results

Result	Measured value
OTIF miss rate in the practice data	median 17.5% (range 15.2%–20.2%), inside the intended 15–25% band on every seed tested
Fused risk model PR-AUC / recall / precision	0.705 / 0.705 / 0.614 (medians, 5 seeds)
Bayesian network PR-AUC (overall / quantity mechanism / timing mechanism)	~0.475 / ~0.417 / ~0.126
Affected-SKU evidence precision / recall	0.536 / 0.627 (5-seed median), versus a naive baseline's 0.09 precision
Decision Value Lab primary gate (50% capacity)	value-aware policy 9.964 vs. shared baseline 9.174 (+8.6%), 4 of 5 seeds won, gate PASSED
Governance lineage	29 checksummed artifacts (pipeline run) / 121 (90-day replay), both fully verified
Offline/batch parity	15/15 orders byte-identical across the adapter boundary
Decision ledger	6,226 decisions logged, 5,923 reconciled against matured outcomes

29.2 Limitations, stated without softening

- All results are measured on **synthetic data** — a realistic simulation, but not a validated stand-in for any real company's supply chain.
- The Bayesian network's standalone accuracy (~0.475 PR-AUC) is meaningfully below XGBoost's (~0.700); it is kept primarily for interpretability and scenario analysis, and its fusion contribution is capped at 10–30%, never forced to an arbitrary equal weight.
- The timing-failure mechanism (`LATE_DELIVERY`) is a weak predictor in this dataset (~0.126 PR-AUC against a ~3.8% base rate) — most modeled misses here are quantity-driven, and timing-side explanations should be read with lower confidence.
- Structural intervention scenarios and evidence attribution are exact computations under a fixed, fitted network's assumptions — never a proven, real-world treatment effect.
- The Decision Value Lab's Bayesian structural-reduction term currently makes its own formula perform *worse*, not better (9.964 with it versus 11.3445 without it) — reported plainly, not hidden, and used directly in Section 24's governance walkthrough.
- Every one of the 8 real model retrains during the 90-day replay was held by the promotion gate, because each one's own held-out accuracy regressed versus the strong initial champion — correct, cautious behavior whose honest cost is a failed rolling-PR-AUC monitoring target.
- The decision ledger's observational cohort comparison (accepted vs. monitored miss rates) is explicitly not a causal estimate — it reflects which orders were prioritized, not a measured effect of acting.
- The AI Copilot's validation guards against presenting unsupported claims as grounded; it does not, and cannot, guarantee an upstream language model can never phrase something misleadingly, only that every cited fact genuinely exists and the persisted decision is preserved.

- Nothing in this system has been tested against a real, live production workload, real network latency, or real concurrent users.

30. How to run it

Full, step-by-step setup instructions for a new machine — including how to download the project without Git and open a command window on macOS or Windows — are kept separately in `docs/setup.md`. The commands below assume that setup is already done.

```
uv sync --extra dev
brew install libomp # macOS only, required for XGBoost

# One scoring run on 2,500 practice orders
uv run otif-risk --orders 2500 --seed 42

# Multi-seed benchmark (the medians/ranges quoted throughout this document)
uv run otif-benchmark --seeds 1 2 3 4 5 --orders 2500 --output-dir artifacts \
  --benchmark-path artifacts/benchmark.json

# Decision Value Lab: 8-policy, 3-capacity-scenario evaluation
uv run otif-policy-benchmark --seeds 1 2 3 4 5 --orders 2500 \
  --benchmark-path artifacts/policy_benchmark.json

# 90-day daily operations replay: scoring, drift, retraining, governance
uv run otif-ops --orders 2500 --seed 42 --replay-days 90 --output-dir artifacts \
  --policy-value-reference-path artifacts/policy_benchmark.json

# The nine-screen interactive control tower
uv run streamlit run src/otif_risk/app.py

# Automated tests
uv run pytest
```

Copy `.env.example` to `.env` only if you want to enable the AI Copilot's live OpenAI mode (Section 25); the Copilot works fully offline with no `.env` file at all.

31. Glossary

As-of timestamp

The exact moment in time a prediction pretends to be made at; only information that genuinely existed by that moment may be used.

Bayesian network

A diagram of cause-and-effect nodes with probability tables, used here for an interpretable, cross-checking risk estimate and "what if" scenario analysis.

Brier score

A number (lower is better) measuring how close predicted probabilities are to what actually happened — a calibration-quality measure.

Calibration

Adjusting a model's raw scores so that, e.g., predictions of "70% risk" really do come true about 70% of the time.

Capacity scenario

A what-if setting (25%/50%/100% of default daily capacity) used to test how policies behave when resources are scarce, not just when they are plentiful.

CPT (Conditional Probability Table)

The table of probabilities attached to one node in a Bayesian network describing how likely it is to be active given its parent nodes' states.

Champion / challenger

The model or policy currently in service (champion) versus a newly proposed replacement (challenger) being evaluated against it.

Content ID

A deterministic fingerprint of a run that ignores timestamps and file paths, so identical runs always produce the same ID.

Data leakage

Accidentally letting a model see information from the future relative to when it is supposed to be making a prediction.

Digital twin

A computer simulation of a real-world system (here, a supply chain) realistic enough to generate practice data with known ground truth.

Do-operator / intervention

A calculation that asks "what would happen if we forced this cause to be resolved," mathematically distinct from simply observing it resolved.

Drift

When real-world patterns change enough that a model trained on older data starts giving worse advice.

Evidence attribution

A leave-one-out sensitivity check on the Bayesian network — not SHAP, and not a causal-effect estimate.

Evidence packet

A small, controlled, cited set of facts about one order, built by plain code, that the AI Copilot is allowed to read and must cite.

Fusion

Blending the XGBoost score and the Bayesian network's score into one final number, using a weight chosen on validation data only.

Idempotency

A property where repeating the same request never creates a duplicate effect — retrying is always safe.

Manifest

A structured record of exactly how a run was produced, including checksums of every output file, for reproducibility and tamper detection.

OTIF (On-Time, In-Full)

An order outcome that requires both arriving on or before the promised date and containing the complete ordered quantity.

PR-AUC (Precision-Recall Area Under the Curve)

A single number (0 to 1, higher is better) summarizing how well a score ranks true positive cases above negative ones.

Precision

Of everything flagged as risky, what fraction genuinely was.

Recall

Of everything that genuinely was risky, what fraction got flagged.

Regret (in the Decision Value Lab)

How far short of the best-possible (oracle) outcome a real, deployable policy falls.

Rolling-origin evaluation

Evaluating a model only on data after its own training cutoff, with that cutoff moving forward through the calendar.

SHAP

A technique for attributing how much each input feature pushed one specific prediction up or down.

SKU (Stock Keeping Unit)

An individual product identifier.

SLO (Service Level Objective)

A transparent, pre-agreed target a metric is expected to meet, checked and reported honestly whether it passes or fails.

Threshold

The risk-score cutoff above which an order is treated as actionable.

XGBoost

A widely-used machine learning technique that combines many small decision trees, each correcting the previous ones' mistakes, into one strong prediction.

32. Appendix: module map and artifact directory map

32.1 Module-to-responsibility map

Code module	Responsibility
<code>data.py</code>	The digital twin: generates orders, lines, events, capacity, shocks, and simulator truth.
<code>validation.py</code>	Fail-fast schema, key, reference, and logical checks (Section 6).
<code>root_causes.py</code>	Retrospective, evaluation/training-label cause derivation from final outcomes.
<code>features.py</code>	Point-in-time feature engineering and the chronological split (Sections 7–9).
<code>line_evidence.py</code>	Capture-time-safe per-line affected-SKU evidence (Section 12).
<code>model.py</code>	The XGBoost risk model (Section 10).
<code>explain.py</code>	SHAP explanations and the deterministic perturbation fallback (Section 11).
<code>bayesian.py</code>	The 10-node mechanism network, inference, structural interventions, and evidence attribution (Sections 13–14).
<code>fusion.py</code>	Validated weight selection and score blending (Section 15).
<code>decisions.py</code> / <code>resources.py</code>	Thresholding, action mapping, resource-aware conflict resolution (Section 16).
<code>narratives.py</code>	Templated, structured planner-facing explanations.
<code>action_response.py</code> / <code>policy_evaluation.py</code> / <code>policy_benchmark.py</code>	The Decision Value Lab (Section 17).
<code>operations.py</code>	The daily operations replay loop (Section 19).
<code>drift.py</code>	Drift detection metrics feeding monitoring/retraining.
<code>benchmark.py</code>	The multi-seed benchmark and acceptance gates.
<code>adapters.py</code> / <code>service_contracts.py</code>	Production-shaped source adapters and score request/response contracts (Section 20).
<code>manifest.py</code>	Deterministic run manifests, content IDs, checksum verification (Section 21).
<code>decision_ledger.py</code>	The decision/outcome ledger and observational cohort reports (Section 22).
<code>monitoring.py</code>	Rolling monitoring and SLO reporting (Section 23).

Code module	Responsibility
<code>registry.py</code>	Champion/challenger versions, promotion gate, lifecycle events (Section 24).
<code>copilot_context.py</code> / <code>copilot_fallback.py</code> / <code>llm_copilot.py</code> / <code>copilot_validation.py</code> / <code>copilot_audit.py</code> / <code>copilot_evaluation.py</code>	The AI Copilot's evidence packets, deterministic fallback, live OpenAI integration, validation, audit log, and evaluation harness (Section 25).
<code>app.py</code>	The nine-screen Streamlit control tower (Section 26).
<code>feedback.py</code>	Append-only planner/system feedback logging.

32.2 Artifact directory map

Path	Contents
<code>artifacts/run-<config-hash>/</code>	One single pipeline run: source tables, simulator truth, feature table, scored orders/lines, fusion comparison, rollups, model files, <code>metrics.json</code> , <code>run_manifest.json</code> , <code>parity_check.json</code> , planner feedback log.
<code>artifacts/ops-<config-hash>/</code>	One operations-replay run: daily queues, model registry, <code>decision_ledger.csv</code> , <code>observational_cohort_report.json</code> , <code>intervention_outcomes.json</code> , <code>monitoring_report.json</code> , <code>demo_lifecycle_scenario.json</code> , a <code>registry/</code> directory (versions, lifecycle events, active-model pointer).
<code>artifacts/benchmark.json</code>	The 5-seed model-quality benchmark (Section 10.6, Section 13.5).
<code>artifacts/policy_benchmark.json</code> + <code>policy_benchmark_manifest.json</code>	The Decision Value Lab's 5-seed, 3-capacity-scenario, 8-policy evaluation (Section 17), plus its own run manifest.
<code>artifacts/copilot_evaluation.json</code>	The AI Copilot's mechanical evaluation report (Section 25.4).
<code>docs/model-card.md</code>	The complete, measured model card with every per-seed table.
<code>docs/demo-script.md</code>	A judge-facing, step-by-step walkthrough of the same reserved scenario used in Section 27.
<code>docs/setup.md</code>	Full new-machine setup instructions.
<code>docs/architecture/current.mmd</code> / <code>current.svg</code>	The project's own generated architecture diagram, rendered by a small, checked-in, deterministic renderer.

End of document. This guide describes a demonstration prototype evaluated entirely on synthetic data; see `docs/model-card.md` for the complete, unabridged measured results and limitations this document has summarized throughout.